

CHATJOEY SPRINT SUMMARY REPORT

From Project Review and Frontend Prototyping to Unified System Integration and Validation

Project	JoeyLLM / ChatJoey
Reporting Period	This Sprint (Three Weeks)
Prepared by	JoeyLLM Project Team
Report Date	10 August 2026
Report Status	English Version - Pending Team Confirmation

Executive Summary

The work completed during this Sprint progressed through three consecutive stages. In Week 1, the team reviewed the project and prepared the working environment. In Week 2, in accordance with Matthew's instructions, each member developed an individual Next.js chat frontend prototype and studied Qdrant and vector databases. In Week 3, the team stopped developing separate versions, compared and consolidated the members' designs, selected YingZhe's interface as the primary template, and completed the integration and deployment of the live JoeyLLM API, followed by independent testing by each team member.

At every stage, the team worked in accordance with Matthew's instructions and the technical resources available at the time. No model fine-tuning, post-training, or other backend experiments were conducted in Week 1 because JupyterHub was under maintenance and the team did not have access to the database, datasets, computing environment, or relevant backend services. This was not the result of the team abandoning or delaying the work.

Core Sprint Outcome: The team established a unified ChatJoey frontend, completed the end-to-end connection and online deployment of the live JoeyLLM API, and had every team member independently test the system connection and model response performance.

Sprint Goal

This Sprint has two primary goals:

Complete the unified ChatJoey frontend and API integration

The team will consolidate the individual frontend designs into a single shared ChatJoey interface. The team will then integrate and deploy the JoeyLLM API and conduct independent end-to-end testing to confirm that the frontend communicates with the API correctly, the system returns responses reliably, and the model's answers align with the team's expectations.

Develop a foundational understanding of vector databases and Qdrant

The team will study the core concepts of vector data, text embeddings, similarity search, and Qdrant. This will prepare the team for the next Sprint, which will focus on vector storage, semantic retrieval, and retrieval-augmented generation.

Sprint Objectives and Scope of Completion

Workstream	Objective	Work Completed	Status
Project Preparation	Restore the team's shared understanding of the previous semester's deliverables, code, and documentation	Reviewed the GitHub repositories and existing materials	Completed
Frontend Development	Create comparable Next.js ChatJoey interface designs	Members completed and presented individual frontend prototypes	Completed
Team Integration	Discontinue competing versions and establish one shared frontend	Selected YingZhe's design as the primary template and improved it collaboratively	Completed
API Integration	Enable the unified frontend to call the live JoeyLLM service	Completed API integration and end-to-end deployment	Completed
System Testing	Confirm online connectivity and observe model response quality	Each member conducted independent question-and-answer testing	Completed
Vector Database Preparation	Develop a foundational understanding of Qdrant and vector retrieval	Completed conceptual study in preparation for the next Sprint	Completed

Week 1: Project Review and Environment Preparation

As not all team members had returned to campus, the team was unable to arrange a full in-person meeting, and Matthew did not assign any new development tasks at that stage. At the same time, JupyterHub and the project server were undergoing maintenance and upgrades, so the team could not access the data, databases, or runtime environment used in the previous semester.

Following the project arrangements available at the time, team members individually accessed GitHub and reviewed and organised the previous semester's repositories, code, documentation, and existing deliverables. This enabled the team to reacquaint itself with the project background, technical direction, and current progress in preparation for subsequent development.

Technical Conditions and Work Boundaries

Until access to the server and database was restored, the team did not have the necessary conditions to conduct model fine-tuning, post-training, or other backend experiments. Such work requires access to training data, the model framework, computing resources, and the runtime environment, none of which had yet been made available to the team.

Note: The scope of work in Week 1 was determined by the project conditions available at the time. The team completed its project review and organised existing materials, creating a shared foundation for subsequent frontend development and system integration.

Week 2: Next.js Frontend Prototypes and Vector Database Study

Matthew asked the team to develop a Next.js-based ChatJoey interface. To explore different design options, each member completed an independent frontend prototype on a personal GitHub branch and presented the interface design and interaction concepts through GitHub and Vercel for subsequent team comparison.

The purpose of the individual prototyping stage was not to maintain multiple products over the long term, but to explore page layouts, chat interactions, responsive design, and deployment approaches before selecting a unified frontend. This stage provided direct reference material for the team's selection and integration work in Week 3.

Qdrant and Vector Database Preparation

Team members also studied the fundamental concepts of vector databases, including text embeddings, vector storage, similarity search, and the role of Qdrant in a retrieval-augmented generation workflow. Qdrant does not generate answers itself; instead, it retrieves content from the project materials that is semantically relevant to the user's question and supplies that content to JoeyLLM.

Conceptual Workflow: Project materials are converted into vectors and stored in Qdrant; the user's question is also converted into a vector; Qdrant returns similar content; and JoeyLLM generates an answer using the retrieved results.

Week 2 Individual Frontend Deliverables

Deliverable Type	Content	Purpose
Next.js Prototype	Independent chat interface, message area, input components, and responsive layout	Compare members' design directions
GitHub Branch	Members committed code to personal branches and created review entry points	Avoid overwriting main directly
Vercel Preview	Deployed individual designs as online previews	Enable the team and stakeholders to review the designs
Technical Study	Studied Qdrant, embeddings, and similarity search	Prepare for retrieval work in the next stage

Week 3: Unified Frontend, API Integration, and System Testing

In response to Matthew's instruction to stop separate development and establish a unified version, the team held a focused discussion on the strengths and weaknesses of each frontend prototype. The team compared page layouts, chat interactions, responsive behaviour, and the feasibility of subsequent API integration.

Following the discussion, the team selected YingZhe's frontend design as the primary template and incorporated suitable elements from other members' designs. Subsequent development centred on this shared version, and the team no longer maintained multiple competing official versions.

Live JoeyLLM API Integration

The team integrated and deployed the live JoeyLLM API within the shared frontend, transforming ChatJoey from a static interface with simulated responses into an end-to-end system capable of sending questions to the model service and displaying genuine responses.

The completed system follows this key workflow: a user enters a question on the ChatJoey page; the Next.js server sends the request to the JoeyLLM API; the API returns the model's response; and the frontend displays the result in the chat message area.

Independent Team Testing

After the API was deployed, each team member independently tested the online system. Members asked JoeyLLM questions from different perspectives to determine whether the webpage could send requests correctly, whether the API returned results reliably, and whether the model's answers aligned with the team's expectations for JoeyLLM's functionality and performance.

The team consolidated and compared the test results to determine whether the end-to-end connection between the frontend and API was functioning correctly, while also identifying possible content deviations, inconsistent responses, or other issues requiring further optimisation.

Testing Principle: A successful API response confirms that the technical deployment and connection are fundamentally complete. Whether the answer meets expectations is used to assess the model's current performance and future optimisation needs. These two findings must be recorded separately and must not be treated as the same conclusion.

Test Dimension	Checks	Completion Criterion
Page Usability	The unified frontend opens online and its input and send functions operate correctly	A user can complete a full question submission
API Connectivity	Requests reach the JoeyLLM API and receive responses	The page reliably displays genuine returned results
Error Handling	Observe interface behaviour during failures, timeouts, or exceptions	Issues can be identified and recorded

Response Quality	Compare relevance, accuracy, and consistency across different questions	Produce observations of model performance
Member Verification	Each member independently tests the system and reports the results	Test results are consolidated by the team

Key Deliverables and Evidence

Deliverable	Evidence	Conclusion
Frontend Prototype Exploration	Personal GitHub branches, code commits, and Vercel previews	Completed
Unified UI Design	Team discussion outcomes and the shared version based on YingZhe's design	Completed
Live API Integration	Live question-and-answer workflow in the unified system	Completed
Online Deployment	Accessible team deployment	Completed
Independent Testing	Questions and response observations recorded by individual members	Completed
Qdrant Study	Understanding of vector database concepts and plans for subsequent work	Foundational preparation completed

Limitations, Dependencies, and Items Outside Scope

The scope completed during this Sprint should be assessed against Matthew's requirements and the access permissions actually available to the team. The team completed the requested frontend prototypes, unified integration, API connection, deployment, and testing activities.

Model fine-tuning, post-training, modification of training data, and database-level backend experiments were not activities that the team was able to perform during this Sprint. At the time, the team did not have access to JupyterHub, the database, training data, computing resources, or the relevant backend services, and therefore could not conduct this work independently.

Limitation or Dependency	Impact	Response
JupyterHub and Server Maintenance	The previous semester's runtime environment was unavailable	Prioritised repository review and frontend preparation
Database Access Not Available	Could not directly inspect or modify the previous semester's data	Await Matthew's access permissions and further instructions
Training Environment Unavailable	Could not conduct fine-tuning or post-training experiments	Did not claim completion of unauthorised backend work
Model Performance Requires Further Observation	Successful API operation does not mean every answer meets expectations	Retain member test records for subsequent optimisation

Sprint Completion Assessment

Across the three weeks, the Sprint completed the full process from project review and individual frontend exploration to team selection, unified system integration, and online validation. The team established a shared ChatJoey frontend, connected and deployed the live JoeyLLM API, and confirmed through independent member testing that the system could perform end-to-end question answering.

The Sprint can therefore be considered to have achieved its core delivery objectives. Model response quality, Qdrant-based retrieval augmentation, data optimisation, and backend experiments requiring server and database access should continue in later Sprints and should not be treated as evidence that the frontend and API deliverables for this Sprint were incomplete.

Completion Conclusion: Within the scope assigned by Matthew and the technical conditions available to the team, the unified ChatJoey frontend, API integration, online deployment, and team testing were all completed.

Recommendations for the Next Stage

Area	Recommended Work	Expected Output
Qdrant	Complete a reproducible example of vector insertion and similarity search	Concept explanation, test data, and retrieval results
RAG	Design a workflow that retrieves project materials before calling JoeyLLM	Basic retrieval-augmented architecture
Model Testing	Create a standardised question set and record response quality	Comparable test results
Data and Training	Develop data-optimisation and training-parameter experiments after access is granted	An experimental plan confirmed by Matthew
Project Documentation	Update the README, deployment URL, operating instructions, and limitations	Reproducible and up-to-date repository documentation